

Experiment – 09

Stored Procedures — PostgreSQL

Name: Samridhi Singh **Section:** 24AIT_KRG-1_A

UID: 24BAI70514 **Semester:** 5th

Branch: BE-CSE(AI ML) **Subject:** Advanced Database Management System

Solutions prepared from the uploaded Experiment 09 material. The experiment covers PostgreSQL stored procedures, OUT parameters, RAISE NOTICE, stock validation, sales insertion, and stock updates.

Problem 1 — Count Employees by Gender

Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter, counts employees belonging to that gender, returns the count through an OUT parameter, and displays the count using RAISE NOTICE.

Table: employee_info (id, name, gender, salary, city)

Solution:

```
CREATE OR REPLACE PROCEDURE pr_count_gender(  
  IN p_gender VARCHAR,  
  OUT employee_count INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  SELECT COUNT(*)  
  INTO employee_count  
  FROM employee_info  
  WHERE gender = p_gender;  
  
  RAISE NOTICE 'Total employees with gender %: %',  
    p_gender, employee_count;  
END;  
$$;
```

Calling the procedure:

```
CALL pr_count_gender('Male', NULL);
```

Explanation: p_gender receives the gender. COUNT(*) calculates the matching employee count. The result is stored in the OUT parameter employee_count, and RAISE NOTICE displays the message.

Problem 2 — Purchase Product Only When Stock Is Sufficient

The procedure accepts a product name and required quantity. It must check available stock. When enough stock exists, it retrieves the product code and price, calculates the total sale amount, inserts an order, decreases remaining stock, and increases quantity sold. Otherwise, no data should be modified.

Tables: products and sales

Solution:

```
CREATE OR REPLACE PROCEDURE pr_buy_products(
  IN p_product_name VARCHAR,
  IN p_quantity INT
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_product_code VARCHAR(10);
  v_price NUMERIC(10,2);
  v_quantity_remaining INT;
  v_total NUMERIC(10,2);
BEGIN
  SELECT product_code, price, quantity_remaining
  INTO v_product_code, v_price, v_quantity_remaining
  FROM products
  WHERE product_name = p_product_name;

  IF NOT FOUND THEN
    RAISE NOTICE 'Product % does not exist.', p_product_name;
    RETURN;
  END IF;

  IF v_quantity_remaining >= p_quantity THEN
    v_total := v_price * p_quantity;

    INSERT INTO sales
    (order_date, product_code, quantity_ordered, sale_price)
    VALUES
    (CURRENT_DATE, v_product_code, p_quantity, v_total);

    UPDATE products
    SET quantity_remaining = quantity_remaining - p_quantity,
        quantity_sold = quantity_sold + p_quantity
    WHERE product_code = v_product_code;

    RAISE NOTICE 'Purchase successful. Total sale amount: %', v_total;
  ELSE
    RAISE NOTICE 'Insufficient quantity available for product %.',
    p_product_name;
  END IF;
END;
$$;
```

How the procedure works

1. Accepts **product_name** and **quantity** as input parameters.
2. Uses SELECT ... INTO to retrieve the product code, price, and current stock.
3. Checks **NOT FOUND** so a missing product does not cause an invalid purchase.
4. Checks whether **quantity_remaining** >= **requested quantity**.
5. If stock is sufficient, calculates **price × quantity**.
6. Inserts the order into the **sales** table.
7. Updates **quantity_remaining** and **quantity_sold** in the products table.
8. Displays a success message using **RAISE NOTICE**.
9. If stock is insufficient, it performs no INSERT or UPDATE and displays an appropriate notice.

Table Structures

Column Name	Data Type	Description
product_code	VARCHAR(10)	Unique product code
product_name	VARCHAR(100)	Product name
price	NUMERIC(10,2)	Price of one unit
quantity_remaining	INT	Current available stock
quantity_sold	INT	Total quantity sold

Column Name	Data Type	Description
order_id	SERIAL	Unique order ID
order_date	DATE	Date on which order was placed
product_code	VARCHAR(10)	Product purchased
quantity_ordered	INT	Quantity purchased
sale_price	NUMERIC(10,2)	Total order amount

Example Calls

For the gender procedure: **CALL pr_count_gender('Male', NULL);**

For the purchase procedure: **CALL pr_buy_products('Laptop', 2);**

The exact sample product values are not specified in the supplied Experiment 09 material, so the example product name is illustrative.

Key Viva Points

What is an OUT parameter? It is used by a PostgreSQL procedure to return a value to the caller.

Why use RAISE NOTICE? To display an informational message while the procedure executes.

Why SELECT ... INTO? It retrieves selected column values and stores them in PL/pgSQL variables.

What does NOT FOUND mean here? It indicates that the SELECT did not find a matching product.

How is sale price calculated? Product price multiplied by quantity ordered.

What happens when stock is insufficient? No INSERT or UPDATE is performed; an insufficient-quantity notice is displayed.